Quy trình xử lý câu trả lời và codebook AUT

Tài liệu đối chiếu triển khai thực tế ngày 01 tháng 10 năm 2026

Tài liệu dành cho người vận hành và phát triển phần mềm AUT tiếng Việt. Phạm vi bao gồm tiếp nhận bài, phân tích ý, truy xuất và sinh mã, phản biện, mở rộng và gộp mã, lưu dữ liệu, chấm điểm, nhận xét, trả kết quả, xử lý lỗi và các công cụ kiểm toán. Mô tả dựa trên mã nguồn đang có trong workspace, kể cả thay đổi chưa commit; không coi các bản kế hoạch cũ là bằng chứng rằng chức năng đã chạy.

Kết luận chính: đây là pipeline nhiều bước có hàng đợi trong PostgreSQL. AI được phép tạo mã được chấp nhận ngay và gộp mã qua kiểm chứng tự động, không cần admin duyệt từng mã. Backend kiểm tra cấu trúc, bằng chứng, tính nhất quán và điều kiện cập nhật; ba chỉ số số lượng, đa dạng, độ hiếm và công thức độ chi tiết do code tính. Những ý chưa chắc chắn được giữ để xử lý tiếp, không đồng nhất với ý không hợp lệ.

Số dòng tham chiếu là tại thời điểm khảo sát mã nguồn và có thể thay đổi khi sửa file. Các tham chiếu ghi đường dẫn tương đối trong repository, tên hàm và dòng bắt đầu; phụ lục liệt kê vị trí để tra cứu. Không đưa khóa API, URL kết nối có mật khẩu hay nội dung cá nhân vào báo cáo. Chỉ đọc cấu hình và ngưỡng đồ vật trong DB; không sửa dữ liệu hay mã nghiệp vụ.

1 Sơ đồ toàn bộ luồng

Người tham gia → xác thực hồ sơ → chọn đồ vật → đồng ý tham gia → phiên 180 giây → nhập tối đa 10 ý → nộp với request_id và survey_session_id → kiểm tra đầu vào và lưu Response QUEUED → worker nhận việc → Extraction → kiểm chứng ý INVALID và sửa contract → truy xuất mã → Curator theo lô → sửa phạm vi mã → Challenger và kiểm chứng từng cặp khi cần → hòa giải các mã mới trong cùng bài → kiểm chứng phạm vi mở rộng → khóa ngắn theo đồ vật và kiểm tra epoch → lưu mã và ResponseIdea → cập nhật centroid và đánh dấu tần suất cần tính lại.

Từ đây có ba nhánh: (a) còn ý chưa mã hóa thì PENDING_REVIEW, resolver tự thử lại; (b) mapping hoàn tất nhưng chưa đủ mẫu thì COLLECTING; (c) đã đủ mẫu thì worker chấm Elaboration, tính các chỉ số, sinh nhận xét, lưu FINAL. Frontend lấy trạng thái nhiều lần và hiển thị kết quả. Khi nhàn, worker còn tính lại tần suất, khôi phục việc hết lease, đánh thức một số ý chưa giải quyết khi có mã mới và chạy audit gộp mã.

Luồng này không phải một lần gọi LLM trả ra toàn bộ điểm. Mapping và scoring vẫn tách biệt. Ngoài hai tầng chính còn có các vòng sửa đầu ra, phản biện, chống xung đột khi nhiều người nộp đồng thời và tác vụ kiểm toán nền.

Tham chiếu: [1] backend/app/controllers/response_controller.py:691 · create_response; [2] backend/app/controllers/response_worker.py:457 · run_one_job; [3] backend/app/pipeline/dynamic_mapping.py:815 · run_idea_extraction; [4] backend/app/pipeline/dynamic_mapping.py:961 · run_code_curator; [5] backend/app/pipeline/scoring.py:182 · run_scoring

2 Đơn vị dữ liệu và các trạng thái

Participant là hồ sơ người tham gia; SurveySession là một phiên có thời điểm bắt đầu và hạn nộp; Response là một lượt nộp độc lập. Một người có thể có nhiều Response, kể cả cùng đồ vật. Response lưu câu gốc, từng dòng nhập, JSON mapping/scoring, metadata, trạng thái xử lý, lease, số lần thử và checkpoint.

ResponseIdea là từng ý sau phân tích, có câu gốc, câu chuẩn hóa, trạng thái, chữ ký chức năng, bằng chứng, embedding, code_id và thông tin phân xử. ID ý được tạo ổn định theo response và vị trí để nối mapping với

scoring. ItemCode là danh mục công dụng, không phải bản thân câu trả lời; nhiều ý khác nhau có thể cùng một mã. PipelineAudit lưu các sự kiện xử lý. Item giữ ngưỡng, codebook_epoch và scores_dirty.

Lớp trạng thái	Giá trị	Ý nghĩa
Trạng thái ý	VALID, INVALID, DUPLICATE	Hiểu được công dụng; không hợp lệ; trùng ý trong cùng bài
Gán mã của ý	ASSIGNED, RESOLVING, UNRESOLVED, NOT_APPLICABLE	Có mã; đang giải quyết; hết lượt tự giải quyết; không cần mã
Phân xử ý	PENDING, RESOLVED	Cần phân xử; đã được admin phân xử
Xử lý bài	QUEUED, RUNNING, SCORING, DONE, FAILED	Chờ; mapping/resolution; chấm; xong việc hiện tại; lỗi dừng
Điểm bài	COLLECTING, PENDING_REVIEW, FINAL	Chưa mở điểm; mapping chưa xong; đã có điểm
Nhãn điểm tương thích	PROVISIONAL, EXCLUDED	Còn trong schema; loại khỏi mẫu. Không phải bước bắt buộc trên luồng chính
Mã	ACCEPTED, REJECTED, UNCERTAIN	Mức chấp nhận mã; độc lập với ACTIVE hoặc MERGED
Vòng đời mã	ACTIVE, MERGED	Mã đang dùng; mã nguồn đã gộp có liên kết tới đích
Đồ vật	COLLECTING, ACTIVE, PAUSED	Thu mẫu; đủ điều kiện; dừng mở điểm

DONE không đồng nghĩa đã có điểm: bài có thể DONE + COLLECTING hoặc DONE + PENDING_REVIEW. VALID cũng không đồng nghĩa đã chấm được: ý còn thiếu code_id vẫn có thể là VALID về nội dung. JSON hiển thị chỉ đánh dấu is_valid để chấm khi ý VALID đã gắn mã. Đây là lý do phải đọc cả trạng thái ý, trạng thái gán mã và trạng thái bài.

Tham chiếu: [6] backend/app/models/models.py:129 · Response; [7] backend/app/models/models.py:249 · ResponseIdea; [8] backend/app/models/models.py:188 · ItemCode; [9] backend/app/pipeline/codebook_service.py:431 · persist_mapping

3 Người tham gia chọn đồ vật và bắt đầu

Email được chuẩn hóa qua schema rồi dùng HMAC để tra cứu; DB lưu bản che và giá trị băm, không lưu email rõ. Hồ sơ có UUID và token truy cập riêng, DB giữ hash của token. API người tham gia kiểm tra X-Participant-Id và, khi bật cấu hình, X-Participant-Token. UUID đúng nhưng token thiếu/sai vẫn bị từ chối. Email tự khai chưa có OTP không phải bằng chứng sở hữu email.

Hồ sơ mới hoặc thiếu thông tin cần hoàn tất profile. Hồ sơ đã tồn tại phải đáp ứng kiểm tra token; xung đột email/UUID trả lỗi thay vì tùy ý đổi danh tính. Admin có thể cấp lại token khôi phục; token cũ mất hiệu lực. Quyền admin dùng JWT riêng và không mượn danh tính participant.

Danh sách hiển thị trên Home được lọc ở frontend: vỏ đạn, kim chỉ, xẻng công binh, mũ cối. Đây không phải danh sách cho phép ở backend. ID cuon_chi vẫn là đồ vật lưu trong DB; tên hiển thị “Kim chỉ” không tự đổi item.name mà pipeline nhận. Người đọc cần phân biệt nhãn giao diện với tên và mô tả đồ vật dùng trong prompt.

POST /api/survey-sessions yêu cầu consent và đồ vật tồn tại. Backend khóa participant, tái sử dụng phiên chưa nộp còn hạn của cùng đồ vật để tránh bấm hai lần tạo hai đồng hồ; nếu không có thì tạo hạn mới sau 180 giây. Thời gian trả về gồm server_now để frontend hiệu chỉnh độ lệch đồng hồ.

Frontend có ba phase READY, ACTIVE, EXPIRED; đồng hồ cập nhật mỗi 250 ms. Bản nháp giữ nội dung, hạn, session_id và request_id trên trình duyệt. Tải lại không có nghĩa được bắt đầu lại 180 giây. Hết giờ, có ít nhất một ý thì tự nộp; trống hoàn toàn không gửi bài. Rời trang trong lúc làm có cảnh báo, nhưng bản nháp trình duyệt chưa phải bài đã lưu ở server.

Tham chiếu: [10] backend/app/controllers/participant_controller.py:57 · identify_participant; [11] backend/app/controllers/participant_controller.py:94 · create_participant; [12] backend/app/core/deps.py:61 · get_participant; [13] backend/app/controllers/survey_controller.py:9 · start_session; [14] frontend/src/lib/survey-items.ts:1; [15] frontend/src/pages/Test.tsx:1

4 Nhận bài và chống nộp trùng

POST /api/score nhận item_id, danh sách responses, request_id, survey_session_id và tùy chọn wait_for_completion. Schema vẫn nhận raw_input cũ rồi tách dòng. Backend bỏ dòng trống, gom khoảng trắng, yêu cầu ít nhất một ý, tối đa 10 ý và tối đa 320 ký tự mỗi ý. Đây là kiểm tra hình thức; chưa đánh giá sáng tạo hay hợp lệ ngữ nghĩa.

Backend khóa đồ vật và kiểm tra request_id theo participant. Nếu lần nộp đã tồn tại với cùng đồ vật và nội dung chuẩn hóa, trả lại Response cũ. Nếu dùng lại request_id nhưng đổi nội dung/đồ vật thì trả 409. Ràng buộc DB và nhánh bắt IntegrityError hỗ trợ chống hai request đồng thời tạo hai bài.

Phiên phải thuộc đúng participant và đồ vật. Phiên đã dùng chỉ cho phát lại cùng bài, không tạo bài mới. Quá deadline cộng survey_grace_seconds bị từ chối; mặc định phần gia hạn truyền bài là 60 giây. Khi bắt buộc phiên mà thiếu session_id thì trả lỗi. Đây là thời gian tiếp nhận của server, không phải cam kết kiểm soát tuyệt đối mọi hành vi chỉnh sửa bên ngoài UI.

Khi hợp lệ, câu gốc được lưu trước khi gọi AI, Response có processing_state=QUEUED, scoring_status=COLLECTING, mapping/scoring trống, nguồn dữ liệu lấy theo survey_data_source. Server commit rồi mới trả mã bài. Việc đóng trang sau khi được chấp nhận không hủy việc trong DB.

Luồng mặc định là bất đồng bộ nếu async_processing_enabled hoặc survey_session_required bật. Tùy chọn chờ ở API chỉ đợi một khoảng cấu hình, không biến toàn bộ hệ thống thành giao dịch LLM đồng bộ. Chỉ khi cả hai cờ đều tắt mới đi nhánh đồng bộ cũ, có mapping, cập nhật điểm và backfill ngay trong controller; đó không phải cấu hình hiện đang dùng.

Tham chiếu: [16] backend/app/schemas/schemas.py:174 · ScoreRequest; [17] backend/app/controllers/response_controller.py:575 · _enqueue_response; [18] backend/app/controllers/response_controller.py:568 · _same_submission; [19] backend/app/controllers/response_controller.py:661 · _wait_for_response_completion; [1] backend/app/controllers/response_controller.py:691 · create_response

5 Worker nhận việc và bảo toàn tiến độ

Lifespan của FastAPI khởi chạy worker khi cần hàng đợi. Worker dùng SELECT FOR UPDATE SKIP LOCKED để mỗi việc được một bên nhận, sắp theo thời điểm tạo và ID. Claim token phân biệt chủ sở hữu lần chạy. Lease giới hạn thời gian sở hữu; heartbeat gia hạn khi việc còn sống. Không giữ transaction DB trong suốt cuộc gọi LLM trên luồng worker chính.

Có ba loại việc: mapping bài mới; resolution cho bài PENDING_REVIEW đến hạn thử lại; scoring cho bài COLLECTING đã ở đồ vật đủ điều kiện và thuộc nguồn dữ liệu đang chấm. Thứ tự ưu tiên luân phiên để giảm việc một loại job chiếm hết lượt. Các nhánh nhận việc còn kiểm tra số lần thử và thời điểm lease/retry.

Worker đọc câu gốc, mã hiện có và epoch, gọi AI ngoài khóa, rồi khóa lại ngắn để lưu. Mọi bước lưu quan trọng kiểm tra claim token vẫn đúng. Nếu worker khác đã nhận lại hoặc bài bị thay đổi, kết quả cũ không được tùy tiện ghi đè.

Extraction checkpoint gắn với hash input_lines. Nếu đầu vào không đổi và extraction hoàn chỉnh, lần chạy lại dùng kết quả đã có; nếu mới xong bước tách mà kiểm chứng INVALID thất bại, có checkpoint trước kiểm chứng để tiếp tục đúng đoạn. needs_extraction_repair báo trường hợp phải làm lại extraction. Resolver ưu tiên chỉ những ý VALID chưa có mã.

Mặc định có 2 worker, poll 2 giây, lease 600 giây, tối đa 3 lần xử lý một job và tối đa 3 vòng resolver. Heartbeat nằm trong khoảng 10–60 giây tùy lease. Đây là giới hạn vận hành, không phải số lần gọi API tối đa của cả bài; bên trong một job còn nhiều stage và repair.

Tham chiếu: [20] backend/app/main.py:13 · lifespan; [21] backend/app/controllers/response_worker.py:43 · _claim; [22] backend/app/controllers/response_worker.py:89 · _heartbeat; [23] backend/app/controllers/response_worker.py:142 · _run_mapping; [24] backend/app/controllers/response_worker.py:491 · start_workers

6 Extraction hiểu ý và xác định hợp lệ

Extraction nhận từng dòng đã làm sạch, tên/mô tả đồ vật, prompt và một số tình huống tham chiếu khi bật. Mỗi dòng phải có một kết quả tương ứng; pipeline hiện không cho LLM tự biến một ô thành nhiều ý được cộng điểm. Câu gốc được ép trở lại đúng dòng nhập, còn normalized chỉ giúp diễn giải rõ hơn.

Mỗi kết quả có line_index, original, normalized, status, uses_target_object, object_used, target_object_role, reason, duplicate_of_index, functional_signature, functional_evidence và inferred_signature_fields. Chữ ký có ba phần lõi goal, object_role, mechanism. Target/context và thông tin biến đổi có thể diễn tả ngữ cảnh nhưng không được biến thành các trường suy luận lõi tùy ý; inferred_signature_fields chỉ nhận ba tên được schema cho phép.

Thiếu một dòng hoặc trả lặp index sẽ kích hoạt sửa riêng dòng đó. Kết quả DUPLICATE phải trỏ tới một ý VALID xuất hiện trước, không trỏ chính nó hoặc dòng phía sau. Tham chiếu sai được hỏi lại; vẫn sai sẽ thành lỗi contract, không được tự cộng điểm.

Trùng ý được xét theo công dụng/vai trò/cơ chế trong cùng lượt. Đổi người dùng, vị trí hay cách diễn đạt không tự làm thành công dụng mới. Ngược lại, hai ý khác nhau cùng được gán một mã không nhất thiết là DUPLICATE: chúng có thể được giữ là hai ý VALID nhưng chỉ tăng một nhóm Flexibility.

Ý VALID cần dùng đúng đồ vật và có vai trò chức năng nhận ra được. Backend có nhánh khôi phục uses_target_object khi model đánh false nhưng các bằng chứng và chữ ký đã thể hiện đúng vật. Nếu vẫn không có căn cứ dùng đúng vật, object guard có thể loại ở bước lưu dù extraction trước đó gọi VALID.

Prompt yêu cầu không tự loại chỉ vì ý nguy hiểm, khác thường, cần biến đổi vật, vật đã qua sử dụng, hoặc có vẻ khó thực hiện. Nhưng không được bịa mục đích/cơ chế để cứu câu vô nghĩa. Một thao tác biến đổi không có công dụng cuối cùng, một vật khác làm toàn bộ việc, hoặc câu không cho biết vai trò nào của vật vẫn có thể không hợp lệ. Đánh giá này còn phụ thuộc diễn giải LLM và kiểm chứng tiếp theo; không phải luật vật lý đã được chứng minh tự động.

Ví dụ chỉ để giải thích quy tắc: “buộc vật lại” có thể đủ công dụng dù ngắn; “cắt ra” chưa nói dùng làm gì cần phân biệt với ý có đích sử dụng; “cho chó ăn” với vỏ đạn không được tự bịa thêm việc chế tạo bát để hợp thức hóa. Những ví dụ này không phải kết quả chạy lại dữ liệu người dùng trong lần phân tích này.

Tham chiếu: [3] backend/app/pipeline/dynamic_mapping.py:815 · run_idea_extraction; [25] backend/app/pipeline/dynamic_mapping.py:65 · _ground_functional_evidence; [26] backend/app/schemas/schemas.py:226 · ExtractedIdea; [27] backend/app/pipeline/prompts/idea_extraction.txt:1

7 Kiểm chứng ý bị loại và sửa contract

Các ý không rỗng mà Extraction gọi INVALID được gửi tới invalid verifier. Mục tiêu là tránh loại nhầm các công dụng khác thường. Bộ kiểm tra yêu cầu đủ đúng tập index, đúng câu nguồn và schema. Nếu đầu ra sai contract, có một vòng repair nhắm vào lỗi thay vì bỏ qua kiểm tra.

Verifier có thể giữ INVALID hoặc đổi thành VALID. Đổi sang VALID chỉ được chấp nhận khi có đúng đồ vật, vai trò và chữ ký lõi đầy đủ. Kết quả không đủ căn cứ được giữ để phân xử; backend không xem bất kỳ câu "hợp lệ" nào từ model là đủ.

Một số cách giải thích INVALID bị bộ dò policy đánh dấu, như chỉ dựa vào "nguy hiểm", "không khả thi", "chỉ là vỏ", "đã được sử dụng". Trường hợp này đi thêm invalid_policy_repair. Nếu xác nhận INVALID bằng lý do không vi phạm cổng policy thì hoàn tất; nếu hợp lệ và qua các cổng thì được phục hồi; nếu vẫn chưa đáng tin thì review_required khiến ý chuyển sang nhánh chưa giải quyết.

Nếu API hoặc contract sau sửa vẫn hỏng, pipeline ném LLMJSONError "Chưa hoàn tất kiểm chứng các ý bị loại; giữ bài để thử lại". Đây là lỗi kỹ thuật của bước kiểm chứng, không có nghĩa người dùng bị trừ điểm vì viết ý INVALID. Worker ghi lỗi và thử lại từ checkpoint; vì thế terminal có thể có traceback mà về sau bài vẫn có kết quả.

Backend đối chiếu functional_evidence với chuỗi nguồn. Bằng chứng không xuất hiện thật bị bỏ hoặc đánh dấu phần suy luận. Cơ chế này chặn trích dẫn bịa, nhưng tự nó không chứng minh toàn bộ suy luận ngữ nghĩa là đúng.

Tham chiếu: [28] backend/app/pipeline/dynamic_mapping.py:595 · _verify_invalid_extractions; [29] backend/app/pipeline/dynamic_mapping.py:758 · _validate_adequacy_result; [30] backend/app/pipeline/dynamic_mapping.py:801 · _finish_extraction; [31] backend/app/pipeline/prompts/idea_invalid_verifier.txt:1; [32] backend/app/pipeline/prompts/idea_invalid_policy_repair.txt:1

8 Truy xuất mã và tình huống tham chiếu

Danh sách mã đưa vào Curator gồm mã ACCEPTED và ACTIVE của đúng đồ vật, đồng thời qua kiểm tra tên/mô tả có nhắc vật mục tiêu. Mã MERGED, REJECTED hoặc UNCERTAIN không phải ứng viên hoạt động bình thường. Điều kiện nhắc tên vật là một bộ lọc triển khai đáng chú ý: mã được lưu cùng item_id nhưng cách đặt tên/mô tả không đáp ứng vẫn có thể vắng khỏi ứng viên.

Embedding tách khỏi model chat. Vector mã hóa phần lõi goal + object_role + mechanism; không dùng tên mã, đối tượng thụ hưởng và bối cảnh để quyết định sự mới của công dụng. Vector mã thiếu hoặc khác model được cập nhật khi truy xuất. Model lưu kèm hậu tố phiên bản biểu diễn core-function-v2 để tránh trộn hai cách mã hóa.

Điểm cấu trúc là giao nhau của token đã chuẩn hóa: 40% goal, 30% object_role, 30% mechanism. Điểm truy xuất = 0,75 × độ tương đồng vector + 0,25 × điểm cấu trúc. Ứng viên được sắp giảm dần theo điểm, hòa thì theo ID. Đây là cách tìm mã đáng xem, chưa phải quyết định cuối cùng MATCH.

Trong caller hiện tại, nếu số mã không quá codebook_full_scan_limit=20 thì đưa toàn bộ vào danh sách ứng viên cho từng ý; lớn hơn thì tối đa novelty_candidate_limit=12. Vì vậy không thể đọc riêng code_candidate_limit=3 rồi kết luận prompt chỉ chứa ba mã. Chú thích cấu hình nói không còn full-scan hiện không khớp caller này.

Centroid/prototype chỉ tham gia truy xuất khi bật centroid_retrieval_enabled, đúng model và mã không bị drift. Cấu hình thực đọc được đang tắt cờ này. Hệ thống vẫn có thể xây centroid để kiểm toán dù chưa dùng nó trong quyết định retrieval.

Tình huống tham chiếu được tải từ bốn file JSON về extraction, matching, creation và rejection. Chọn theo stage bằng embedding, có điểm cộng cho cùng đồ vật; mặc định tối đa hai case. Đầu vào prompt được nén theo ngân sách ký tự; case đầu tiên vẫn có thể vượt mức mềm vì chỉ dừng thêm từ case sau. Đây là ví dụ về quy tắc, không phải mã thật để gắn vào ResponseIdea.

Provider embedding chạy theo lô mặc định 64 văn bản, chuẩn hóa vector, kiểm tra đủ kết quả, số hữu hạn và norm khác 0; có ba lần thử. local-hash-v1 là nhánh mock/suy giảm được khai báo, không nên gọi nó là semantic embedding tương đương mô hình cloud. Lỗi embedding cloud không tự chuyển âm thầm thành một vector ngữ nghĩa giả.

Tham chiếu: [33] backend/app/pipeline/codebook_service.py:41 · list_curator_codes; [34] backend/app/pipeline/code_retrieval.py:99 · rank_code_candidates; [35] backend/app/pipeline/code_retrieval.py:67 · signature_similarity; [4] backend/app/pipeline/dynamic_mapping.py:961 · run_code_curator; [36] backend/app/pipeline/reference_dataset.py:114 · reference_cases_json; [37] backend/app/pipeline/embedding.py:62 · embed_texts

9 Curator đề xuất gán hoặc sinh mã

Chỉ ý VALID có uses_target_object đi vào Curator. Không có ý phù hợp thì bỏ qua Curator, Challenger và hòa giải mã mới. Bài nhiều ý được chia lô theo mapping_batch_size=3, dùng chung kết quả truy xuất để giảm việc lặp lại, rồi ghép kết quả và hòa giải cuối cùng.

Curator nhận câu gốc, câu chuẩn hóa, vai trò của vật, chữ ký, bằng chứng, phần suy luận và danh sách ứng viên kèm phạm vi. Kết quả gồm quyết định, mã hiện có hoặc mô tả mã đề xuất, quan hệ phạm vi, confidence và lý do. Một quyết định cho mỗi index là bắt buộc; index lạ/lặp bị kiểm soát, ý thiếu được hỏi riêng.

Quyết định	Cách xử lý thực tế
MATCH_EXISTING	Đề nghị gắn mã đang có; phải qua kiểm tra ID, quan hệ phạm vi và tín hiệu
CREATE_NEW	Đề nghị sinh danh mục công dụng mới; phải có phạm vi tái sử dụng và qua các cổng backend
EXPAND_EXISTING	Đề nghị mở rộng phạm vi mã đích, có thể hấp thụ mã khác; cần kiểm chứng thành viên
OUT_OF_CODEBOOK	Chưa khớp mã; đi phản biện hoặc chuyển sang chưa giải quyết
UNCERTAIN	Chưa đủ căn cứ; không ép vào mã gần nhất
INVALID	Nếu mâu thuẫn với extraction VALID, được chuyển sang chưa chắc chắn để xử lý tiếp

Nếu cả batch lỗi JSON/schema, code hiện hỏi lại từng ý bằng prompt ngắn, kiểm tra kết quả và gom các quyết định phục hồi. Nếu chỉ thiếu vài ý thì chỉ gọi lại các index thiếu. Không phải mọi lỗi đều gọi lại nguyên bài ngay lập tức. Những ý không phục hồi được vẫn phải đi nhánh chưa chắc chắn.

Backend hydrate thông tin từ ý gốc, không tin model được phép đổi vật hoặc câu nguồn. Mã phân loại chức năng bỏ target/context khỏi phần phạm vi cốt lõi. Mục đích tạo mã phải có căn cứ ở câu gốc, không chỉ xuất hiện trong phần model suy diễn. Các trường policy_gates là bằng chứng hỗ trợ; điều kiện quyết định thực sự nằm ở các cổng backend, không phải cứ JSON tự ghi true là được lưu.

Tham chiếu: [4] backend/app/pipeline/dynamic_mapping.py:961 · run_code_curator; [38] backend/app/pipeline/dynamic_mapping.py:467 · _normalize_curator_contract; [39] backend/app/pipeline/dynamic_mapping.py:220 · _hydrate_curator_decision; [40] backend/app/pipeline/dynamic_mapping.py:131 · _guard_ungrounded_creations; [41] backend/app/pipeline/prompts/code_curator.txt:1

10 MATCH và phản biện quyết định

MATCH thông thường cần mã thuộc tập ứng viên hợp lệ, confidence tối thiểu 0,5, hạng truy xuất đầu tiên và quan hệ SAME_CATEGORY hoặc IDEA_NARROWER_THAN_CODE. “Ý hẹp hơn mã” được chấp nhận vì mã là nhóm bao quát công dụng; “ý rộng hơn mã” không được lặng lẽ xem như MATCH.

Cổng mặc định	MATCH ban đầu	MATCH sau phản biện
Điểm truy xuất tối thiểu	0,60	0,50
Tương đồng vector tối thiểu	0,70	0,70
Chênh lệch với mã khác	0,08	0,08
Điểm goal và cấu trúc	Mỗi loại ít nhất 0,20	Có thể chấp nhận diễn đạt đồng nghĩa

Chênh lệch thấp có ngoại lệ khi bằng chứng chức năng tách ứng viên rõ: goal đủ cao và dẫn trước đủ lớn, hoặc tổng cấu trúc đủ cao và dẫn trước. Các ngưỡng hiện thực là goal ≥ 0,40 và lợi thế goal ≥ 0,25 cùng cổng cấu trúc, hoặc cấu trúc ≥ 0,45 và lợi thế ≥ 0,20. Cần đọc snapshot bằng chứng để biết ý đã dùng nhánh nào.

Nếu có candidate_evaluations theo contract cũ, đánh giá phải nhất quán: đúng quan hệ, MATCH, không vướng loại trừ, goal đúng và ít nhất vai trò/cơ chế phù hợp, không có nhiều ứng viên cùng được nhận MATCH. Lệch câu nguồn hoặc mâu thuẫn đánh giá làm quyết định cần xem lại.

Challenger nhận những quyết định cần phản biện cùng câu gốc, chữ ký, đề xuất và ứng viên. Tập được phản biện là OUT_OF_CODEBOOK, UNCERTAIN, EXPAND_EXISTING và INVALID; CREATE_NEW trực tiếp không nằm trong tập này. MATCH không đủ cổng được đưa về UNCERTAIN trước để xem lại. Challenger không phải người duyệt admin và vẫn có thể dùng cùng model với Curator nếu không cấu hình riêng.

Challenger lỗi hoặc bỏ sót ý có vòng bổ sung; không trả được quyết định đáng tin thì giữ UNCERTAIN. Nếu cố MATCH một mã đúng ID nhưng tín hiệu còn yếu, có thể gọi pair verifier. Pair verifier yêu cầu câu nguồn khớp và quote có thật. Quan hệ SAME_CATEGORY/IDEA_NARROWER_THAN_CODE cùng goal và vai trò phù hợp, không vướng exclusion cho phép đặt pair_verified để vượt các cổng số phù hợp; kết quả này vẫn là một phán đoán LLM có bằng chứng, không phải xác nhận khoa học tuyệt đối.

Nếu kiểm chứng cặp cho thấy IDEA_BROADER_THAN_CODE và vai trò phù hợp, hệ thống có thể gọi propose_scope để sinh EXPAND_EXISTING, thay vì ép vào mã hẹp hoặc lập một mã gần trùng. Nếu vẫn thiếu căn cứ, kết quả là chưa giải quyết. Cờ pair_verified do backend kiểm soát, không nhận tùy ý từ JSON của Curator.

Tham chiếu: [42] backend/app/pipeline/dynamic_mapping.py:311 · _match_signal_reason; [43] backend/app/pipeline/dynamic_mapping.py:295 · _low_margin_is_resolved_by_function; [44] backend/app/pipeline/dynamic_mapping.py:1737 · run_code_challenger; [45] backend/app/pipeline/semantic_verifier.py:13 · verify_pair; [46] backend/app/pipeline/semantic_verifier.py:31 · propose_scope

11 CREATE_NEW sinh mã như thế nào

Mã mới không chỉ là một nhãn ngắn. Đề xuất cần tên, mô tả, chữ ký đủ ba phần, quy tắc bao gồm và loại trừ cụ thể, ví dụ dương và phạm vi có thể tái sử dụng. Các scope_variants cần cho thấy nhiều biến thể chức năng trong cùng phạm vi, không chỉ lặp câu nguồn; cổng reusable_scope yêu cầu ít nhất hai biến thể chuẩn hóa khác nhau và không dùng target/context để tách nhóm.

Các quy tắc quá chung như “có cơ chế phù hợp” không đủ để phân ranh giới. Mã thiếu hoặc có boundary chung chung được hỏi repair. Repair không được tùy ý đổi loại quyết định hoặc đổi mã đích của EXPAND. Sửa xong vẫn thiếu căn cứ thì UNCERTAIN, không tạo một mã nửa vời để lấy điểm.

CREATE_NEW có thể do Curator đề xuất trực tiếp mà không gọi Challenger, hoặc được Challenger đề xuất từ một ca chưa chắc chắn. Cả hai đều qua cổng vai trò của vật, mục đích được câu gốc hỗ trợ, tính mới và phạm vi. Quan hệ tạo mới phải phù hợp DIFFERENT hoặc NOT_APPLICABLE. Khi commit, kiểm tra trùng normalized_name hoặc functional_key với mã hoạt động. Va chạm không có bằng chứng dùng chung proposal_group khiến ý bị giữ lại để giải quyết, không tự tạo bản sao.

Mã được chấp nhận được lưu ItemCode với UUID, item_id, tên chuẩn hóa, chữ ký, functional_key, inclusion_rules, exclusion_rules, positive_examples, embedding/model, confidence, lý do và nguồn response. Luồng AI đặt ACCEPTED + ACTIVE ngay; không đợi admin xác nhận. Tạo mới làm tăng codebook_epoch và ý được gắn code_id.

Hàm tìm/tạo có các nhánh tương thích cho tên đã có, mã đã gộp, dữ liệu chữ ký cũ còn thiếu và một số mã chưa được chấp nhận nhưng không admin_locked. Tuy nhiên không nên hiểu là mọi đề xuất va tên đều được tái sử dụng: persist_mapping có cổng va chạm trước đó. Cần phân biệt hàm hỗ trợ có khả năng làm gì với trường hợp caller thực sự cho nó chạy.

Tham chiếu: [47] backend/app/pipeline/dynamic_mapping.py:1305 · _has_reusable_scope; [48] backend/app/pipeline/dynamic_mapping.py:1331 · _repair_missing_boundaries; [49] backend/app/pipeline/codebook_service.py:109 · _find_or_create_code; [9] backend/app/pipeline/codebook_service.py:431 · persist_mapping

12 Chống mã mới trùng nhau trong cùng bài

Một bài có nhiều CREATE_NEW được so các chữ ký để phát hiện khả năng chồng lấn. Cổng sơ bộ gồm vai trò giao nhau đủ lớn với goal/cơ chế, hoặc tổng cấu trúc đủ lớn với vai trò/cơ chế. Đây là kiểm tra đề xuất mới với nhau, bổ sung cho truy xuất mã đã có trong DB.

Nếu có chồng lấn, LLM reconciliation phải chia đúng các index vào nhóm và trả mã đại diện có phạm vi tái sử dụng. Mỗi index xuất hiện đúng một lần. Các đề xuất cùng nhóm được thống nhất tên, mô tả, chữ ký, ví dụ và proposal_group để lúc lưu chỉ tạo một mã cho nhóm.

Hai đề xuất còn chồng lấn nhưng tên khác nhau cần có quyết định nhóm khác nhau rõ ràng; nếu không thì những ý liên quan thành UNCERTAIN. Reconciliation lỗi chỉ làm các đề xuất nguy cơ chồng lấn cần giải quyết, không mặc nhiên loại mọi ý còn lại.

Sau hòa giải, hai ý VALID chung mã vẫn là hai ý nếu Extraction không xác định DUPLICATE. Ví dụ bài có hai biến thể công dụng khác nhau trong cùng phạm vi danh mục: Fluency tăng theo hai ý, Flexibility chỉ tính một code_id. Tần suất của mã trong mẫu tăng theo số ý VALID, không theo một lần xuất hiện duy nhất trong bài.

Tham chiếu: [50] backend/app/pipeline/dynamic_mapping.py:1483 · _potential_new_code_overlaps; [51] backend/app/pipeline/dynamic_mapping.py:1511 · _reconcile_new_code_batch; [9] backend/app/pipeline/codebook_service.py:431 · persist_mapping

13 Mở rộng mã và gộp tự động

EXPAND_EXISTING là thay đổi phạm vi của mã đích đang hoạt động; có thể kèm absorbed_code_ids. Đề xuất phải thể hiện ý rộng hơn mã cũ, qua phản biện, có boundary cụ thể và quan hệ hấp thụ phù hợp. Đây là nhánh có thay đổi ý nghĩa danh mục, không chỉ sửa tên hiển thị.

Trước khi ghi, scope audit lấy các câu đang thuộc mã đích và các mã bị hấp thụ, thêm câu hiện tại làm trường hợp dương; các ví dụ dương của mã khác được dùng làm trường hợp âm. Từng trường hợp cần quote thật và kết luận quan hệ đúng. Ý cũ phải vẫn thuộc phạm vi; ví dụ âm không được bị hút vào phạm vi mới một cách không phân biệt.

Mặc định tối đa 80 trường hợp, chia lô 8. Nếu số trường hợp quá giới hạn hoặc một kiểm chứng không đạt/API lỗi thì đề xuất thành UNCERTAIN, không sửa phạm vi. Khi sắp commit, guard_members kiểm tra tập thành viên có thay đổi từ lúc audit không; thay đổi thì không nhận kết quả audit cũ.

Khi đạt, mã đích cập nhật phạm vi và lưu scope_history, tăng scope_revision, cập nhật embedding và epoch. Các ResponseIdea của mã nguồn được chuyển code_id sang mã đích. Mã nguồn giữ lại trạng thái MERGED và merged_into_id để truy vết; không bị xóa cứng. Nếu tên chuẩn hóa va với mã không nằm trong tập hấp thụ thì không chấp nhận. JSON mapping của các bài liên quan được đồng bộ và tần suất cần tính lại.

Audit nền còn có thể gộp những mã đã có. Khi không có job ưu tiên, worker xét cặp mã theo tín hiệu lexical/semantic, hạn chế số lịch sử thử, lấy ví dụ nguồn kiểm chứng với đích, rồi kiểm chứng toàn bộ phạm vi trước khi gộp. Nó không chỉ thấy cosine cao là đổi code_id ngay. Trình tự ứng viên theo ID hiện tại không đồng nghĩa mã tạo trước luôn là mã đích.

Audit có trạng thái RUNNING/DONE/RETRY và lease; epoch cũ hoặc lỗi sẽ không cho ghi kết quả gộp. MOCK_MODE hoặc tắt auto_code_audit_enabled thì nhánh này không chạy. admin_locked không phải một khóa bất biến tổng quát được kiểm tra ở mọi nhánh mở rộng tự động; không nên diễn giải cờ đó như bảo đảm mã không bao giờ đổi.

Không thấy một luồng tự tách mã thành nhiều mã con đang thực thi trong các đường gọi khảo sát đã kiểm tra. Công cụ cluster audit chỉ gợi ý cụm, không phải thao tác split hay merge tự động.

Tham chiếu: [52] backend/app/pipeline/scope_audit.py:16 · verify_expansions; [53] backend/app/pipeline/scope_audit.py:81 · guard_members; [54] backend/app/pipeline/codebook_service.py:284 · _expand_existing_code; [55] backend/app/controllers/code_audit_worker.py:54 · run_one_audit

14 Chống xung đột và lưu mapping

Trong lúc AI suy nghĩ, một người khác có thể tạo/gộp mã của cùng đồ vật. Worker vì vậy không lưu mù theo codebook cũ: nó kiểm tra codebook_epoch dưới khóa đồ vật. Nếu thay đổi, đọc lại mã rồi chạy lại Curator trên extraction đã có, tối đa số vòng recheck cấu hình. Hết vòng vẫn biến động thì chuyển lỗi để thử lại sau.

Ngay cả khi cần xem xét tạo mới, guard_stale_creations kiểm tra mã mới xuất hiện so với tập đã đối chiếu. Tên chuẩn hóa và functional_key là hai lớp chặn va chạm khi commit. Đây là cơ chế giảm trùng do đồng thời, không chứng minh mọi công dụng đồng nghĩa đều có cùng chuỗi functional_key.

persist_mapping xử lý từng ý: INVALID/DUPLICATE không được gắn mã chấm; VALID không qua object guard có thể thành INVALID; thiếu quyết định, quyết định chưa chắc hoặc không qua cổng sinh/gán mã thì giữ ý chờ mã. review_required của Extraction đi nhánh EXTRACTION_REVIEW. MATCH/CREATE/EXPAND đạt mới có ASSIGNED.

Ý chưa gán mã lưu review_payload gồm đề xuất và bằng chứng để resolver/admin đọc. Worker lưu mapping_meta, checkpoint, codebook_epoch, những code_id đã so sánh và audit MAPPING_COMMIT. Bài có ý

chờ mã trở thành PENDING_REVIEW. Nếu không còn ý chờ, mapping hoàn tất và bài về COLLECTING để chờ điều kiện chấm.

Resolver chỉ thay những index cần sửa, không tùy ý xóa các ý đã hoàn tất. ID ổn định giữ sự liên kết với lần phân tích trước. Khi mở rộng mã, mapping của các bài ảnh hưởng được đồng bộ; sau thay đổi đặt scores_dirty để tránh hiển thị điểm đang không khớp tần suất/mapping.

Centroid của mã được xây lại từ các vector thành viên hợp lệ có thể dùng. Hệ thống lưu số lượng, tổng vector, centroid, model và prototype; phát hiện khác model/dimension hoặc lệch nhiều thì gắn drift_flag. Prototype gồm đại diện gần tâm và các đại diện xa hơn để mô tả phạm vi. Drift không tự có nghĩa loại mã hay tách mã, và cờ này không tự mất chỉ vì lần tính sau có vẻ ổn.

Tham chiếu: [56] backend/app/controllers/response_worker.py:104 · _guard_stale_creations; [23] backend/app/controllers/response_worker.py:142 · _run_mapping; [9] backend/app/pipeline/codebook_service.py:431 · persist_mapping; [57] backend/app/pipeline/codebook_service.py:399 · synchronize_response_mappings; [58] backend/app/pipeline/centroid.py:59 · rebuild_confirmed_members

15 Ý chưa giải quyết và phân xử admin

Sau mapping còn ý VALID thiếu mã, worker hẹn resolver theo resolver_retry_seconds, mặc định 15 giây. Resolver tự xử lý lại với codebook hiện có, không bắt buộc admin duyệt. Sau tối đa resolver_max_attempts, ý chuyển UNRESOLVED và bài vẫn PENDING_REVIEW; giao diện không nên quay vòng vô hạn như một job đang chạy bình thường.

Có một cơ chế đánh thức bổ sung khi codebook_epoch tăng: xét bài đã hết resolver nhưng chưa được đánh thức, so chữ ký ý chờ với các mã mới chưa từng đối chiếu. Nếu có mã đủ liên quan theo cổng cấu trúc thì cho một lượt giải quyết thêm và ghi dấu đã dùng cơ hội này. Không có mã liên quan thì chỉ cập nhật epoch đã xem. Không phải cứ có bất kỳ mã mới nào là toàn bộ bài cũ được gửi lại LLM.

Admin có API phân xử một ý PENDING: MATCH_EXISTING, CREATE_NEW hoặc đánh INVALID. Backend khóa item và ý, kiểm tra vẫn PENDING, từ chối sửa mapping bài FINAL và có thể kiểm tra expected_codebook_epoch. MATCH cần mã đích ACCEPTED + ACTIVE; CREATE cần tên, mô tả, chữ ký ba phần, inclusion/exclusion và không va tên/functional_key.

Admin tạo mã từ hàng đợi được ghi created_by=ADMIN, confidence=1, admin_locked và thông tin người/thời gian duyệt. Embedding ở nhánh này dùng local hash; đây là điểm khác đường AI cloud cần chú ý khi kiểm toán model vector. Quyết định admin được lưu vào mapping_evidence và review_resolution, cập nhật centroid nếu có mã.

Sau phân xử, scoring cũ của bài bị xóa để tính lại; còn ý chờ thì PENDING_REVIEW, hết ý chờ thì COLLECTING, processing_state=DONE. Các claim/error được reset. Phân xử admin là lối xử lý ngoại lệ đang có, không phải bước bắt buộc của mọi mã AI tạo.

Tham chiếu: [59] backend/app/controllers/response_worker.py:278 · _wake_one_unresolved; [23] backend/app/controllers/response_worker.py:142 · _run_mapping; [60] backend/app/controllers/admin_controller.py:1071 · resolve_mapping_review

16 Điều kiện mở điểm và mẫu tính tần suất

Mẫu hiệu chỉnh gồm bài đúng nguồn dữ liệu, không PENDING_REVIEW, không EXCLUDED, đã DONE hoặc SCORING, có ý VALID gắn mã ACCEPTED + ACTIVE. Một ý chưa giải quyết khiến cả Response không đủ điều kiện tham gia mẫu theo truy vấn hiện tại; không chỉ bỏ riêng ý chưa có mã.

Số người dùng ngưỡng là số participant khác nhau có ít nhất một ý đủ điều kiện. Một người nộp nhiều bài vẫn là một người cho ngưỡng, nhưng mọi ý VALID của các bài đủ điều kiện đều tăng mẫu tần suất. Đồ vật khác nhau tính mẫu riêng, không lấy số người/ý toàn hệ thống để mở điểm cho từng đồ vật.

Đồ vật được chấm khi không PAUSED và đạt đồng thời ngưỡng số người và số ý hợp lệ. Ngưỡng lấy từ Item, có fallback cấu hình. Mặc định cấu hình là 24 người và 150 ý. Đọc DB ngày 01/10/2026 cho thấy bốn đồ vật đang hiển thị đều đặt 5 người, 150 ý, trạng thái ACTIVE. Chỉ đủ năm người nhưng tổng ý chưa đến 150 vẫn chưa mở điểm.

Nguồn hiện đang cấu hình PILOT. calibration_source_filter cho phép bài PILOT và dữ liệu SYNTHETIC có metadata calibration_source=PILOT. Khi chuyển SURVEY, dữ liệu mô phỏng không được tự coi là dữ liệu khảo sát thực trong mẫu này. Các corpus đã sinh theo độ hiếm dự kiến là phân bố mô phỏng phục vụ thử nghiệm, không phải tần suất thực nghiệm đo trên quần thể.

Khi đồ vật vượt ngưỡng, worker có thể nhận chấm các bài COLLECTING trước đó; vì vậy người vận hành thấy nhiều bài cũ lần lượt có điểm là đúng với cách chọn việc. Không phải người dùng phải nộp lại từng bài. Nếu mẫu tụt dưới ngưỡng, item quay lại COLLECTING trừ PAUSED; cách ẩn điểm đã có còn phụ thuộc cờ dirty và các đường cập nhật, không nên suy ra mọi nhãn FINAL đều bị xóa ngay.

Tham chiếu: [61] backend/app/pipeline/codebook_service.py:792 · calibration_source_filter; [62] backend/app/pipeline/codebook_service.py:803 · _eligible_response; [63] backend/app/pipeline/codebook_service.py:851 · item_is_ready_for_scoring; [64] backend/app/pipeline/codebook_service.py:862 · refresh_item_scoring_state; [21] backend/app/controllers/response_worker.py:43 · _claim

17 Tính bốn chỉ số

Fluency đếm các ý VALID. Trên luồng chuẩn, bài đang thiếu mã bị giữ PENDING_REVIEW nên chưa được chấm; hàm tính riêng vẫn có khả năng đếm VALID chưa mã. Flexibility là số code_id hoạt động khác nhau trong bài, không phải số chữ ký khác nhau và không phải số ô có chữ.

Originality dựa vào từng ý: p(code) = số ý VALID thuộc mã trong mẫu đủ điều kiện của đồ vật / tổng số ý VALID đã mã hóa trong mẫu đó. p ≤ 1% cho 2 điểm; 1% < p ≤ 5% cho 1 điểm; p > 5% cho 0 điểm. Tổng bài là tổng điểm các ý được chấm. Đây là độ hiếm trong mẫu đang dùng, không phải model tự ước lượng “ý này rất sáng tạo”.

Ví dụ với mẫu 150 ý: mã có 1 ý chiếm khoảng 0,67% và được 2 điểm mỗi ý; mã có 2–7 ý được 1 điểm; mã có từ 8 ý chiếm trên 5% nên 0 điểm. Nhiều ý cùng mã trong các bài vẫn đóng góp nhiều đơn vị vào tử số. Không dùng số người có mã chia số người tham gia.

Elaboration được tính từ bốn nhóm chi tiết target, mechanism, context, goal. LLM chỉ trả present và đoạn bằng chứng cho từng nhóm theo đúng idea_id và original. Backend kiểm tra không thiếu/trùng ID, không lệch câu nguồn; chuẩn hóa nhẹ chữ hoa/khoảng trắng/dấu câu để xác nhận quote có trong câu thật. Không có bằng chứng được chấp nhận thì nhóm đó không cộng điểm.

Mỗi ý có Elaboration = 1 + số nhóm có bằng chứng, nên từ 1 đến 5. Nếu scoring_runs > 1, một nhóm phải được đa số các lượt đồng ý bằng quote có căn cứ; mặc định hiện tại là một lượt. Số từ có nghĩa được tính riêng bằng stoplist và bỏ từ tên vật để làm đối chứng, không thay công thức điểm.

Tổng Elaboration của Response là tổng các ý, nên có thể lớn hơn 5. “Tối đa 5” chỉ đúng cho từng ý hoặc trung bình mỗi ý. Admin hiển thị trung bình tổng điểm giữa các bài cũng có thể vượt 5. Do đó giá trị 6,75 không thể kết luận sai chỉ từ việc lớn hơn 5; cần biết UI đang lấy tổng, trung bình bài hay trung bình ý.

Không có ý để chấm thì bỏ LLM scoring và phản hồi phù hợp: không có ý hợp lệ, hoặc có ý hợp lệ nhưng chưa đủ căn cứ gắn mã. Việc có kết quả 0 không đồng nhất với lỗi kỹ thuật. Bài có toàn ý INVALID vẫn được lưu mapping và có thể nhận kết quả khi đồ vật đã đáp ứng điều kiện chấm.

Tham chiếu: [65] backend/app/pipeline/codebook_service.py:873 · originality_for_response; [66] backend/app/pipeline/scoring.py:92 · _aggregate_facets; [5] backend/app/pipeline/scoring.py:182 · run_scoring; [67] backend/app/pipeline/prompts/scoring.txt:1

18 Lưu điểm và tính lại khi mẫu thay đổi

Worker đọc cơ sở điểm rồi gọi LLM bên ngoài transaction. Trước khi lưu, nó kiểm tra lại item còn đủ điều kiện không và tập ý còn đúng không. Nếu mapping thay đổi trong lúc chấm thì không ghép nhầm bằng chứng vào ý khác; trả việc về DONE để vòng sau xử lý lại khi còn COLLECTING.

Khi tập ý vẫn khớp, worker tính lại Fluency/Flexibility/Originality theo DB mới nhất, ghép Elaboration đã lấy từ LLM bằng idea_id, lưu scoring, các cột tổng, scored_at, metadata và frequency_basis. Bài chuyển FINAL, processing_state=DONE, xóa claim/error và reset attempts.

Khi mapping/mã thay đổi, scores_dirty khiến API tạm không trả scoring để tránh hiển thị điểm cũ. Tác vụ recount cập nhật trạng thái item và điểm tần suất, rồi xóa dirty. refresh_final_frequency_scores duyệt bài FINAL thuộc nguồn hiện tại, tính lại các chỉ số phụ thuộc mẫu, giữ Elaboration, evidence và summary đã có khi nối được đúng ý.

Nếu xuất hiện ý hiện tại chưa có điểm cũ hoặc câu nguồn không còn khớp thì bài về COLLECTING và xóa scoring để chấm lại phần cần thiết. Nếu chỉ tỷ lệ mã thay đổi, không cần gọi LLM chấm Elaboration lại. Summary định tính được giữ, vì contract hiện tại không phụ thuộc tần suất.

Do đó FINAL ở code hiện hành có nghĩa đã hoàn tất chấm theo cơ sở được ghi, không bảo đảm bất biến suốt khảo sát. codebook_epoch và scope_revision là bộ đếm kiểm soát thay đổi; frequency_basis là bản ghi cơ sở tính điểm. Chúng không tạo một hệ thống nhiều phiên bản codebook mà người dùng/admin chọn qua lại.

Tham chiếu: [68] backend/app/controllers/response_worker.py:314 · _run_scoring; [69] backend/app/controllers/response_worker.py:265 · _recount_one_item; [70] backend/app/pipeline/codebook_service.py:954 · refresh_final_frequency_scores; [71] backend/app/controllers/response_controller.py:542 · _to_response

19 Nhận xét định tính và hiển thị kết quả

creative_feedback chạy sau tính điểm, nhận tên đồ vật và original của các ý được chấm, không nhận độ hiếm để suy đoán năng lực. Prompt hiện yêu cầu một nhận định ngắn về cách phát triển công dụng, một lời mời làm rõ/mở rộng ý có sẵn, khách quan, không kết luận tính cách/IQ/năng lực cố định và không viết sẵn công dụng mới thay người dùng.

Contract có đúng một observation, text tối đa 450 ký tự, một hoặc hai evidence với quote tối đa 160 ký tự, suggestion tối đa 200 ký tự và tổng không quá 80 từ theo bộ đếm của code. Bài từ hai ý phải có hai idea_id khác nhau làm bằng chứng; bài một ý chỉ nhận xét ý đó. Quote phải là đoạn thật trong đúng original; backend tự ghép “Ví dụ” và chỉ một nhãn “Bạn có thể thử”.

Nếu API, schema, quote, độ dài hoặc file prompt có lỗi, dùng nhận xét dự phòng bằng code, ghi status=FALLBACK; không làm mất điểm hay buộc chạy lại cả scoring. Nếu không có ý, bỏ qua. Prompt feedback đọc lại theo lần gọi; sửa file không tự viết lại summary đã lưu của các bài cũ. Các prompt khác có template nạp ở cấp module, nên cần chú ý vòng đời tiến trình khi sửa.

Trong Test, sau submit thành công UI giữ trạng thái đang phân tích và poll từ 750 ms tăng đến tối đa 4 giây, giới hạn chờ 180 giây. Khi hoàn tất mới chuyển trang Result. Quá thời gian hoặc mất mạng có thể hiện đề nghị kiểm tra/gửi lại; request_id cũ giúp trả về bài đã lưu thay vì tạo thêm bài giống hệt.

Result có navigation state và sessionStorage cache để mở nhanh, sau đó lấy API tùy nguồn mở. Poll thường tăng đến 10 giây; khi lỗi mạng có backoff đến 30 giây và sau sáu lần hiện kết nối lại. QUEUED/RUNNING/SCORING, resolution_pending hoặc scores_stale khiến UI tiếp tục chờ. FAILED hiện thông báo bài đã lưu nhưng cần admin xử lý, không yêu cầu làm lại nội dung.

DONE + COLLECTING đã ổn định có thể hiển thị trạng thái thu mẫu chứ không quay AI thinking mãi. PENDING_REVIEW hết resolver không còn là một tác vụ đang tiến triển vô hạn. Lần mở từ lịch sử có nhánh tải trạng thái mới; cache chỉ là tối ưu hiển thị, không phải nguồn tính điểm. Người tham gia chỉ đọc được bài thuộc hồ sơ mình; admin đọc qua endpoint riêng.

Tham chiếu: [72] backend/app/pipeline/creative_feedback.py:37 · generate_creative_feedback; [73] backend/app/pipeline/prompts/creative_feedback.txt:1; [74] backend/app/pipeline/scoring.py:148 · _summary; [15] frontend/src/pages/Test.tsx:1; [75] frontend/src/pages/Result.tsx:1; [76] frontend/src/lib/response-status.ts:1

20 Lỗi API retry và khôi phục

chat_json gửi prompt, model, temperature, JSON mode, ngân sách output và reasoning_effort phù hợp provider. Nếu provider không hỗ trợ response_format hoặc reasoning_effort thì bỏ tham số đó và gọi lại theo nhánh tương thích. Nếu có thông báo context phù hợp mẫu nhận diện, có thể giảm output token để vừa context nhưng vẫn giữ mức tối thiểu.

Đầu ra được parse trực tiếp, từ code fence hoặc phần JSON trong văn bản. finish_reason cho thấy chạm giới hạn output là lỗi riêng, không coi JSON cắt dở là kết quả hợp lệ. Schema ở từng stage tiếp tục kiểm tra sau bước parse; JSON hợp lệ về cú pháp chưa chắc đúng contract.

Lỗi tạm thời có retry và thời gian đợi ưu tiên Retry-After/reset-token, giới hạn 30 giây mỗi lần. Quota/balance hết hoặc lỗi HTTP 400/401/403/404/422 được đánh dấu không nên retry theo logic wrapper; 429 đơn thuần không tự có nghĩa hết tiền. Metadata gồm stage, prompt_hash, model, attempts, usage, latency, fallback và loại lỗi; lưu raw response là tùy chọn đang mặc định tắt.

Khi một job lỗi, worker ghi PIPELINE_FAILURE, giữ tối đa năm failure metadata gần nhất và chỉ lưu loại exception trong processing_error để hạn chế lưu lại nội dung cá nhân do provider lặp ra. Job còn lượt và retryable thì hẹn lại, mặc định backoff 60 giây × số lần, tối đa 300 giây. Hết lượt hoặc không retryable thì FAILED. Traceback terminal là nhật ký lỗi lần chạy, không nhất thiết trạng thái cuối bài.

Worker chết/hết lease có thể được nhận lại. Nếu RUNNING/SCORING hết lease và hết attempts thì _expire_exhausted đặt FAILED với WorkerLeaseExpired. Claim token ngăn tiến trình cũ quay lại ghi sau khi mất quyền sở hữu. Checkpoint giúp tránh lặp extraction đã xong.

Admin retry_failed_response chỉ nhận bài FAILED; reset attempts/lease/error, về QUEUED nếu chưa có mapping hoặc DONE nếu có mapping để scheduler chọn công việc phù hợp. Nó không tự thay đổi câu trả lời và cũng không bảo đảm mọi loại dữ liệu bất nhất đều tự lành sau một lần bấm.

Hết quota được ghi error_category=QUOTA_EXHAUSTED và dashboard tìm trong mapping_meta/scoring_meta để đếm bài bị ảnh hưởng. Đây là phát hiện từ lỗi API đã xảy ra, không phải API truy vấn số dư liên tục hay dự báo chính xác token còn lại. Lỗi feedback quota có thể chỉ đưa feedback về FALLBACK trong khi điểm vẫn hoàn tất. Embedding có lớp lỗi riêng, không có mọi metadata quota giống chat_json.

Tham chiếu: [77] backend/app/pipeline/llm.py:172 · chat_json; [78] backend/app/pipeline/llm.py:31 · quota_exhausted; [79] backend/app/controllers/response_worker.py:401 · _record_failure; [80] backend/app/controllers/response_worker.py:437 · _expire_exhausted; [81] backend/app/controllers/admin_controller.py:141 · retry_failed_response; [82] backend/app/controllers/admin_controller.py:329 · _has_quota_failure

21 Admin loại gộp xóa mã thực sự làm được gì

Phải phân biệt ba việc: loại một ý do nội dung không hợp lệ; từ chối đề xuất mã vì thiếu căn cứ; và thay đổi mã đã tồn tại. INVALID của ý không bắt buộc tạo một ItemCode REJECTED. Đề xuất CREATE bị chặn thường khiến ý chờ phân xử, không đồng nghĩa toàn bộ công dụng đó đã bị cấm vĩnh viễn.

Các hàm patch_code, merge_code, delete_code, delete_all_codes và remap_item hiện có raise HTTPException 409 ở đầu nhánh. Phần cập nhật còn nằm phía dưới là code không tới được trong đường gọi này. Router admin đang được công khai trong file cũng không khai báo các endpoint sửa/gộp/xóa cũ ấy. Vì vậy không mô tả các chức năng đó như thao tác admin đang hoạt động chỉ vì tìm thấy tên hàm hoặc code phía dưới.

Admin hiện có các endpoint dashboard, danh sách participant, cấp lại token, xem bài, retry bài FAILED, xem codebook, extraction/curator audit, pipeline audit, mapping review, cluster audit và export. Phân xử ý PENDING vẫn hoạt động và có thể tạo mã hoặc loại ý theo mục 15.

Cluster audit gom ý chưa gán mã thành cụm gợi ý bằng complete-linkage: mọi cặp trong cụm phải đủ gần, cùng model/dimension và có từ goal liên quan. Nó trả ứng viên mã gần, margin và độ phân tán; không tự gắn mã hay thay điểm. Mã thiếu vector vẫn được báo, không giả vờ đã được phân cụm đầy đủ.

Nhánh AI EXPAND_EXISTING và auto code audit vẫn có thể gộp và chuyển code_id như mục 13, dù admin merge bị chặn. Đây là sự khác nhau thực tế giữa quyền thao tác của admin và tự động hóa hiện hành. Câu “sổ mã append-only/bất biến” trong thông báo cũ cần được đọc trong giới hạn endpoint đó, không dùng để mô tả toàn hệ thống.

Tham chiếu: [83] backend/app/controllers/admin_controller.py:1411 · patch_code; [84] backend/app/controllers/admin_controller.py:1491 · merge_code; [85] backend/app/controllers/admin_controller.py:1601 · delete_code; [86] backend/app/controllers/admin_controller.py:1655 · delete_all_codes; [87] backend/app/controllers/admin_controller.py:1281 · remap_item; [88] backend/app/routers/admin.py:1; [89] backend/app/pipeline/cluster_proposals.py:66 · propose_clusters

22 Ma trận các tình huống cần phân biệt

Tình huống	Kết quả hoặc nhánh tiếp theo
Không có nội dung	Schema từ chối; chưa tạo bài
Trên 10 ý hoặc ý quá 320 ký tự	Schema từ chối
Thiếu/sai danh tính hoặc token	400/401/403 tùy bước kiểm tra
Đồ vật không tồn tại	404
Phiên sai chủ hoặc sai đồ vật	Từ chối phiên
Nộp sau hạn và thời gian gia hạn	409; không tạo bài mới
Gửi lại cùng request_id và nội dung	Trả Response đã có
Cùng request_id nhưng đổi nội dung	409
Hết giờ có ý	Frontend tự nộp

Tình huống	Kết quả hoặc nhánh tiếp theo
Hết giờ không có ý	Không tạo bài rỗng
Extraction thiếu/lặp dòng	Sửa theo dòng/index
DUPLICATE trỏ sai	Repair; vẫn sai thì lỗi contract
INVALID rõ nghĩa và được kiểm chứng	Giữ INVALID; không mã hóa/chấm ý
INVALID bị loại nhầm	Verifier có thể phục hồi VALID qua cổng
Verifier trả sai schema	Repair rồi retry job nếu chưa xong
Ý VALID không dùng đúng vật	Object guard có thể chuyển INVALID
Có mã khớp đủ căn cứ	MATCH_EXISTING rồi ASSIGNED
Mã gần nhưng tín hiệu mâu thuẫn	Challenger/pair verifier hoặc UNCERTAIN
Công dụng mới đủ phạm vi	Tạo ACCEPTED + ACTIVE tự động
Tên/chữ ký mã mới va mã đã có	Chưa giải quyết hoặc tái đối chiếu
Hai đề xuất mã mới chồng lấn	Reconciliation nhóm và tạo mã đại diện
Hai ý khác nhau chung mã	Có thể Fluency 2 và Flexibility 1
Ý rộng hơn mã hiện có	Đề xuất scope và kiểm chứng EXPAND
Audit phạm vi không đạt hoặc quá giới hạn	Không mở rộng; giữ chưa chắc chắn
Gộp tự động đạt kiểm chứng	Chuyển thành viên, nguồn MERGED, đích cập nhật
Codebook đổi trong lúc gọi AI	Kiểm tra epoch và chạy lại Curator
Có ý chưa mã hóa	Cả bài PENDING_REVIEW; chưa vào mẫu đủ điều kiện
Resolver hết lượt	UNRESOLVED; có thể admin hoặc một lần đánh thức
Chưa đủ người hoặc số ý	DONE + COLLECTING
Vượt ngưỡng sau bài mới	Bài cũ đủ điều kiện được scheduler chấm
Mẫu tần suất đổi sau FINAL	Tính lại phần tần suất, giữ Elaboration nếu nối đúng
Mapping đổi lúc LLM chấm	Không lưu kết quả gắn sai ý; chờ vòng sau
Nhận xét lỗi nhưng điểm xong	Summary FALLBACK, không mất điểm
Lỗi mạng/rate limit tạm thời	Retry có giới hạn

Tình huống	Kết quả hoặc nhánh tiếp theo
Hết quota hoặc lỗi không retryable	Có thể FAILED ngay; admin thấy dấu lỗi
Worker chết hoặc mất lease	Nhận lại hoặc FAILED khi hết attempts
Mất mạng frontend sau server đã nhận	Bài vẫn trong DB; lấy lại bằng ID/request_id
Admin phân xử ý đang PENDING	Gán/tạo/loại ý và xếp lại trạng thái bài
Admin sửa/gộp/xóa mã qua hàm cũ	Bị chặn 409; không mô tả như tính năng đang mở
Mock hoặc dữ liệu fixture	Không đại diện chất lượng LLM thực

23 Cấu hình thực đọc được và điểm cần lưu ý

Thông số	Giá trị đọc ngày 01/10/2026
MOCK_MODE	false
Nguồn khảo sát	PILOT
Xử lý bất đồng bộ	Bật
Bắt buộc phiên và token participant	Bật cả hai
Ngưỡng mặc định cấu hình	24 người và 150 ý
Ngưỡng DB bốn đồ vật đang hiển thị	5 người và 150 ý mỗi đồ vật
Trạng thái bốn đồ vật	ACTIVE tại lúc đọc
Số lượt scoring	1
Tự audit mã	Bật
Truy xuất bằng centroid	Tắt
Lô Curator	3 ý
Full scan và ứng viên khi vượt ngưỡng	Tối đa 20 mã thì toàn bộ; lớn hơn lấy 12

Các ngưỡng retrieval và confidence là cổng kỹ thuật cấu hình, chưa phải chứng nhận độ đúng trên khảo sát thực. Báo cáo này phân tích các đường đi trong code và đọc ngưỡng DB, không chạy lại mọi tổ hợp lỗi, không gọi LLM trên toàn bộ câu trả lời riêng tư và không phải kết quả kiểm định tâm lý học.

Có các khác biệt cần giữ trong tài liệu vận hành: FINAL vẫn cập nhật tần suất; admin merge bị khóa nhưng AI merge còn chạy; centroid được xây nhưng hiện chưa bật vào retrieval; số ứng viên thực khác chú thích cấu hình; tên hiển thị Kim chỉ và item cuon_chi cần thống nhất ý nghĩa đồ vật. Test.tsx còn có một câu giới thiệu “1 cuộn chỉ mới cứng...” viết trực tiếp trong giao diện, cần lưu ý khi mở bài đồ vật khác.

Ngoài ra, scope audit chọn thành viên theo mã và trạng thái ý, không áp dụng đầy đủ bộ lọc nguồn khảo sát như mẫu tính điểm. Centroid và tần suất cũng không có bộ lọc hoàn toàn giống nhau ở mọi caller. Vì vậy một vector đại diện hoặc tập kiểm chứng phạm vi không nên được diễn giải là chính xác cùng một tập dữ liệu với frequency_basis.

Để kiểm tra một bài cụ thể, nên lần theo response_id → input_lines và session/request → processing_state/attempts/failures → extraction checkpoint và bằng chứng → quyết định Curator/Challenger cùng retrieval snapshot → code_id/scope_revision/epoch → review payload nếu có → frequency_basis → evidence Elaboration → metadata feedback → JSON API/cache giao diện. Cách này phân biệt được lỗi hiểu ý, lỗi gán mã, lỗi mẫu tính điểm và lỗi nhận xét/hiển thị.

Tham chiếu: [90] backend/app/config.py:11 · Settings; [91] backend/app/pipeline/scope_audit.py:10 · member_rows; [92] backend/app/controllers/admin_controller.py:48 · pipeline_audits; [93] backend/app/controllers/admin_controller.py:86 · get_cluster_audit

24 Phạm vi đã đối chiếu và cách sử dụng báo cáo

Các nhóm đã đối chiếu gồm frontend nhập bài và đọc kết quả; API xác thực participant và session; schema đầu vào/đầu ra; controller nhận bài; worker mapping/resolution/scoring; prompt và contract extraction; truy xuất embedding; Curator/Challenger/repair/reconciliation; scope và pair verifier; lưu mã/ý; centroid và audit nền; mẫu hiệu chỉnh; chấm điểm; creative feedback; retry/quota; phân xử/admin và các nhánh cũ bị chặn.

Các bản kế hoạch và chú thích nghiệp vụ giúp hiểu mục tiêu, nhưng khi chúng khác code thì báo cáo ưu tiên điều kiện thực thi. Danh mục tình huống bao phủ các nhánh nghiệp vụ và lỗi rõ trong những đường gọi đã kiểm tra; không thể xem đó là chứng minh đã kiểm thử mọi tổ hợp đồng thời, mọi phản hồi provider hay mọi trường hợp ngôn ngữ tự nhiên.

Tài liệu không sửa thiết kế trong lần này. Những mâu thuẫn được nêu để người vận hành đọc đúng hệ thống và làm căn cứ cho lần chỉnh sửa tiếp theo. Phụ lục sau tự tra vị trí hàm từ mã nguồn để giảm sai số tham chiếu thủ công.

Phụ lục tra cứu mã nguồn

Mỗi số tham chiếu dưới đây tương ứng với file và dòng bắt đầu của hàm hoặc lớp. Các đường dẫn tính từ thư mục gốc repository. Tham chiếu toàn file dùng dòng 1. Số dòng được tra tự động từ cây cú pháp Python, không suy đoán từ bản kế hoạch.

[1] backend/app/controllers/response_controller.py:691 · create_response

[2] backend/app/controllers/response_worker.py:457 · run_one_job

[3] backend/app/pipeline/dynamic_mapping.py:815 · run_idea_extraction

[4] backend/app/pipeline/dynamic_mapping.py:961 · run_code_curator

[5] backend/app/pipeline/scoring.py:182 · run_scoring

[6] backend/app/models/models.py:129 · Response

[7] backend/app/models/models.py:249 · ResponseIdea

[8] backend/app/models/models.py:188 · ItemCode

[9] backend/app/pipeline/codebook_service.py:431 · persist_mapping

[10] backend/app/controllers/participant_controller.py:57 · identify_participant

[11] backend/app/controllers/participant_controller.py:94 · create_participant
[12] backend/app/core/deps.py:61 · get_participant
[13] backend/app/controllers/survey_controller.py:9 · start_session
[14] frontend/src/lib/survey-items.ts:1
[15] frontend/src/pages/Test.tsx:1
[16] backend/app/schemas/schemas.py:174 · ScoreRequest
[17] backend/app/controllers/response_controller.py:575 · _enqueue_response
[18] backend/app/controllers/response_controller.py:568 · _same_submission
[19] backend/app/controllers/response_controller.py:661 · _wait_for_response_completion
[20] backend/app/main.py:13 · lifespan
[21] backend/app/controllers/response_worker.py:43 · _claim
[22] backend/app/controllers/response_worker.py:89 · _heartbeat
[23] backend/app/controllers/response_worker.py:142 · _run_mapping
[24] backend/app/controllers/response_worker.py:491 · start_workers
[25] backend/app/pipeline/dynamic_mapping.py:65 · _ground_functional_evidence
[26] backend/app/schemas/schemas.py:226 · ExtractedIdea
[27] backend/app/pipeline/prompts/idea_extraction.txt:1
[28] backend/app/pipeline/dynamic_mapping.py:595 · _verify_invalid_extractions
[29] backend/app/pipeline/dynamic_mapping.py:758 · _validate_adequacy_result
[30] backend/app/pipeline/dynamic_mapping.py:801 · _finish_extraction
[31] backend/app/pipeline/prompts/idea_invalid_verifier.txt:1
[32] backend/app/pipeline/prompts/idea_invalid_policy_repair.txt:1
[33] backend/app/pipeline/codebook_service.py:41 · list_curator_codes
[34] backend/app/pipeline/code_retrieval.py:99 · rank_code_candidates
[35] backend/app/pipeline/code_retrieval.py:67 · signature_similarity
[36] backend/app/pipeline/reference_dataset.py:114 · reference_cases_json
[37] backend/app/pipeline/embedding.py:62 · embed_texts
[38] backend/app/pipeline/dynamic_mapping.py:467 · _normalize_curator_contract
[39] backend/app/pipeline/dynamic_mapping.py:220 · _hydrate_curator_decision
[40] backend/app/pipeline/dynamic_mapping.py:131 · _guard_ungrounded_creations
[41] backend/app/pipeline/prompts/code_curator.txt:1
[42] backend/app/pipeline/dynamic_mapping.py:311 · _match_signal_reason
[43] backend/app/pipeline/dynamic_mapping.py:295 · _low_margin_is_resolved_by_function
[44] backend/app/pipeline/dynamic_mapping.py:1737 · run_code_challenger
[45] backend/app/pipeline/semantic_verifier.py:13 · verify_pair
[46] backend/app/pipeline/semantic_verifier.py:31 · propose_scope
[47] backend/app/pipeline/dynamic_mapping.py:1305 · _has_reusable_scope
[48] backend/app/pipeline/dynamic_mapping.py:1331 · _repair_missing_boundaries
[49] backend/app/pipeline/codebook_service.py:109 · _find_or_create_code
[50] backend/app/pipeline/dynamic_mapping.py:1483 · _potential_new_code_overlaps
[51] backend/app/pipeline/dynamic_mapping.py:1511 · _reconcile_new_code_batch
[52] backend/app/pipeline/scope_audit.py:16 · verify_expansions
[53] backend/app/pipeline/scope_audit.py:81 · guard_members
[54] backend/app/pipeline/codebook_service.py:284 · _expand_existing_code
[55] backend/app/controllers/code_audit_worker.py:54 · run_one_audit
[56] backend/app/controllers/response_worker.py:104 · _guard_stale_creations
[57] backend/app/pipeline/codebook_service.py:399 · synchronize_response_mappings
[58] backend/app/pipeline/centroid.py:59 · rebuild_confirmed_members
[59] backend/app/controllers/response_worker.py:278 · _wake_one_unresolved

[60] backend/app/controllers/admin_controller.py:1071 · resolve_mapping_review
[61] backend/app/pipeline/codebook_service.py:792 · calibration_source_filter
[62] backend/app/pipeline/codebook_service.py:803 · _eligible_response
[63] backend/app/pipeline/codebook_service.py:851 · item_is_ready_for_scoring
[64] backend/app/pipeline/codebook_service.py:862 · refresh_item_scoring_state
[65] backend/app/pipeline/codebook_service.py:873 · originality_for_response
[66] backend/app/pipeline/scoring.py:92 · _aggregate_facets
[67] backend/app/pipeline/prompts/scoring.txt:1
[68] backend/app/controllers/response_worker.py:314 · _run_scoring
[69] backend/app/controllers/response_worker.py:265 · _recount_one_item
[70] backend/app/pipeline/codebook_service.py:954 · refresh_final_frequency_scores
[71] backend/app/controllers/response_controller.py:542 · _to_response
[72] backend/app/pipeline/creative_feedback.py:37 · generate_creative_feedback
[73] backend/app/pipeline/prompts/creative_feedback.txt:1
[74] backend/app/pipeline/scoring.py:148 · _summary
[75] frontend/src/pages/Result.tsx:1
[76] frontend/src/lib/response-status.ts:1
[77] backend/app/pipeline/llm.py:172 · chat_json
[78] backend/app/pipeline/llm.py:31 · quota_exhausted
[79] backend/app/controllers/response_worker.py:401 · _record_failure
[80] backend/app/controllers/response_worker.py:437 · _expire_exhausted
[81] backend/app/controllers/admin_controller.py:141 · retry_failed_response
[82] backend/app/controllers/admin_controller.py:329 · _has_quota_failure
[83] backend/app/controllers/admin_controller.py:1411 · patch_code
[84] backend/app/controllers/admin_controller.py:1491 · merge_code
[85] backend/app/controllers/admin_controller.py:1601 · delete_code
[86] backend/app/controllers/admin_controller.py:1655 · delete_all_codes
[87] backend/app/controllers/admin_controller.py:1281 · remap_item
[88] backend/app/routers/admin.py:1
[89] backend/app/pipeline/cluster_proposals.py:66 · propose_clusters
[90] backend/app/config.py:11 · Settings
[91] backend/app/pipeline/scope_audit.py:10 · member_rows
[92] backend/app/controllers/admin_controller.py:48 · pipeline_audits
[93] backend/app/controllers/admin_controller.py:86 · get_cluster_audit